Scalar SMU: Selective Recurrence Acceleration for Native Online Attention on a RISC-V Vector Cluster

Xiaoteng Wang

***Abstract*—Native online attention processes score tiles incrementally, but each noninitial tile updates a running maximum and normalizer through a state-dependent scalar recurrence. We study how to map that recurrence in a programmable RISC-V vector cluster, where the accompanying output update exposes D-element parallelism. We design a cluster-local Scalar Online-Merge Unit (SMU) for the recurrence and retain the existing Spatz/RVV datapath for the output update. Two custom-2 instructions expose the hardware: `OMCFG` writes nine persistent context fields and commits initialization, while `OMERGE` processes each later score tile across all configured N query rows and returns completion status. Hardware tracks alternating scalar-state buffers; software retains output-vector ownership. Across four native workloads (tile size four, seed one), the MMIO-controlled selective implementation yields 8.524–17.784× speedup in the recurrence window, 4.144–6.275× in the inclusive merge window, and 1.066–1.157× in the Native Core window over a matched scalar-software recurrence with the same RVV update. Native Core excludes Q/K/V loading and linear projections; it is not a full-attention or full-model measurement. The mixed-precision SMU reduces standalone mapped area by 27.613% relative to the legacy scalar implementation, while its pre-layout delay proxy is higher.**

***Index Terms*—RISC-V, online attention, hardware/software co-design, domain-specific acceleration, vector processing, mixed-precision computing, Softmax.**

I. Introduction

Transformer attention combines matrix multiplication with Softmax normalization and is a fundamental operation in modern sequence models [1]. Online normalization updates running statistics as scores arrive [2]. FlashAttention applies this idea to tiled exact attention and avoids materializing the complete attention matrix through IO-aware execution [3]; FlashAttention-2 further improves GPU parallelism and work partitioning [4], and FlashAttention-3 extends the line with asynchronous scheduling and low-precision execution [5]. In a programmable processor, the online formulation also creates a loop-carried recurrence between score tiles.

Each merge combines two different kinds of work. A scalar recurrence updates the running maximum and normalizer and produces two output-rescaling weights. This recurrence forms a dependency chain whose operation count does not grow with the output dimension D. The output update applies those weights to D elements independently. Thus, one merge couples $O(1)$ dependency-constrained scalar operations with $O(D)$ regular vector operations. The design question is where to place the boundary between a dedicated unit and a programmable vector datapath: a uniform offload duplicates vector resources, whereas leaving the recurrence on the scalar core preserves the dependency chain.

Prior work already establishes online normalization and specialized Softmax hardware. Softermax combines online normalization and low-precision arithmetic for hardware-friendly Softmax [6]; SoftEx accelerates Softmax and GELU beside a RISC-V cluster [7]. VEXP adds approximate exponentiation to Snitch RISC-V execution and evaluates both Softmax and FlashAttention-2 [8]. Other work reuses Softmax hardware for GELU [9]. RISC-V platforms also span scalar cores with accelerator interfaces [10], vector processors [11], and shared-memory vector clusters [12]. TOP-V shows that a dependency-heavy operation can be specialized inside a vector processor while its surrounding computation remains programmable [13]. Its configurable sorting unit handles Top-K; the unit boundary studied here lies inside the native online-attention merge.

We study that partition on a Spatz RISC-V vector cluster. A cluster-local Scalar Online-Merge Unit (SMU) executes only the state-dependent recurrence; the existing RISC-V Vector (RVV) datapath retains the D-dimensional output update. The SMU uses fixed, stage-aware mixed precision across the recurrence and keeps memory-visible state in FP32. Software controls it through two implementation-specific custom-2 instructions. `OMCFG` programs persistent TCDM addresses and the query-row count N and required addresses, then commits the context; `OMERGE` processes one later score tile across all N configured rows and returns completion status. Hardware selects the alternating scalar-state buffers, while software owns tile scheduling, output-buffer pointers, and the RVV update. This interface gives the RISC-V core explicit control of the accelerator without adding an SMU-specific vector instruction or a second D-wide datapath.

Across four native online-attention workloads, the matched recurrence window improves by up to 17.8× relative to scalar-software recurrence with the same RVV output update; the broader Native Core window improves by at most 1.16× and excludes Q/K/V loading and linear projections. It is not a full-attention or model-execution result. Separately, the mixed-precision SMU reduces standalone mapped area against the legacy scalar unit, with a higher pre-layout delay proxy. The instruction comparisons show large reductions in their measured recurrence/control and setup windows but no material Native Core improvement.

The paper makes three contributions, ordered by the primary claim:

- We characterize the scalar/vector asymmetry inside native online-attention merging and evaluate a selective HW/SW partition: the Scalar SMU accelerates the state-dependent recurrence, while the existing RVV datapath retains the

regular D-dimensional output update. Section IV-B reports the measured cycle-count ratios for these three scopes.

- We define a RISC-V programming model for the selective partition. `OMCFG` configures nine persistent scalar-context fields and commits initialization; `OMERGE` processes one later score tile across the configured N query rows and returns status. The hardware adapter tracks scalar A/B state and routes completion to the issuing core, while software retains the output buffers and uses existing RVV instructions for the vector update.
- We implement a cluster-local, fixed stage-aware mixed-precision SMU using FP16, lookup-based fixed-point nonlinear approximation, and Q16.32 normalization accumulation behind an FP32 memory-visible interface. Its standalone mapped area is lower than the legacy scalar design; the measured timing proxy is higher, and cluster-level PPA remains unavailable.

II. Native Online Attention and Design Motivation

A. Native Online Attention

Throughout, N is the configured query-row count, B the key-tile size, and $T = N/B$ the number of tiles for the divisible workloads studied here; D is the output-vector length and t indexes a tile. The word *tile* in a weight subscript marks its source contribution. A prime marks merged state and a hat a value after hardware conversion. In a $\mathrm{Q}I.F$ format, I counts integer magnitude bits and F fractional bits; signed formats add one sign bit. Attention aggregates value vectors by query-key similarity. A conventional implementation materializes scores and normalized probabilities before accumulating values. Online attention instead processes score tiles and updates the normalization and output states incrementally, so the full probability matrix is never formed. This streaming normalizer underlies online softmax [2] and IO-aware exact attention [3].

For a query row, let s_i denote the score and $\mathbf{v}_i$ the associated value vector at each position i processed so far. The accumulated state is $(m, l, \mathbf{O})$, with the invariant of Eq. (1):

$$\begin{aligned} m &= \max_i s_i, \\ l &= \sum_i \exp(s_i - m), \\ \mathbf{O} &= \frac{1}{l} \sum_i \exp(s_i - m)\mathbf{v}_i. \end{aligned} \tag{1}$$

Thus, l is the softmax denominator relative to the running maximum m, and $\mathbf{O} \in \mathbb{R}^D$ is the corresponding normalized partial output. A new score tile supplies state $(m_t, l_t, \mathbf{O}_t)$.

The exact scalar merge combines the two states without reconstructing earlier probabilities:

$$\begin{aligned} m' &= \max(m, m_t), \\ l' &= l \exp(m - m') + l_t \exp(m_t - m'), \\ w_{\mathrm{old}} &= \frac{l \exp(m - m')}{l'}, \qquad w_{\mathrm{tile}} = \frac{l_t \exp(m_t - m')}{l'}. \end{aligned} \tag{2}$$

The vector output state then follows Eq. (3):

$$\mathbf{O}' = w_{\mathrm{old}}\mathbf{O} + w_{\mathrm{tile}}\mathbf{O}_t. \tag{3}$$

The first tile initializes the running state directly. Each later tile evaluates the recurrence and updates the D-element output. Because the output remains normalized, no final normalization pass is needed.

B. Execution Characteristics

Fig. 1 separates the merge into a state-dependent scalar recurrence and a regular vector update. The recurrence updates m, l, and the two weights; the vector path applies them independently to D output elements.

The recurrence limits intra-merge data parallelism. State carries across tiles, and maximum selection, exponential rescaling, accumulation, and normalization form a heterogeneous dependency chain. Its scalar work is independent of D, so increasing D adds no recurrence lanes.

Each output element j obeys the opposite pattern:

$$\mathbf{O}'[j] = w_{\mathrm{old}}\mathbf{O}[j] + w_{\mathrm{tile}}\mathbf{O}_t[j], \qquad 0 \le j < D, \tag{4}$$

All D elements share the same weights and have no inter-element dependency. The update is a regular floating-point multiply-add loop and maps naturally to the existing RISC-V Vector (RVV) execution resources. Per query row and per tile, the recurrence performs a fixed number of scalar operations independent of D, while the output update applies two weighted products to each of the D elements.

The accelerator therefore targets the recurrence alone. Score generation, tile-local state construction, and the D-dimensional output update stay programmable, and only the part whose parallelism does not grow with D moves into hardware. For a configured N-row tile merge, the engine repeats this $O(1)$-in-D recurrence once per row; RVV applies the weights to the D output elements of each row.

C. Design Objectives

Three objectives follow from these execution characteristics.

The first is selective specialization: dedicate hardware to the recurrence and reuse Spatz/RVV for the regular output update. That preserves RVV programmability without duplicating a D-wide datapath.

The second is a recurrence unit light enough for cluster integration. Its stages have different numerical requirements, which Section III addresses with stage-aware mixed precision.

The third is a HW/SW interface that lets software configure workload-persistent context once and invoke recurring merges while hardware tracks the alternating recurrence state.

In the resulting mapping, a cluster-local mixed-precision Scalar Online-Merge Unit (SMU) executes the recurrence, the Spatz RISC-V vector datapath performs the D-dimensional output update, and hardware tracks the active A/B selector for TCDM-resident recurrence state.

III. Architecture and HW/SW Co-Design

A. System Overview and HW/SW Partition

Figure 2 shows how the design fits into a Spatz vector cluster [12]. The Snitch scalar core and the Spatz/RVV datapath

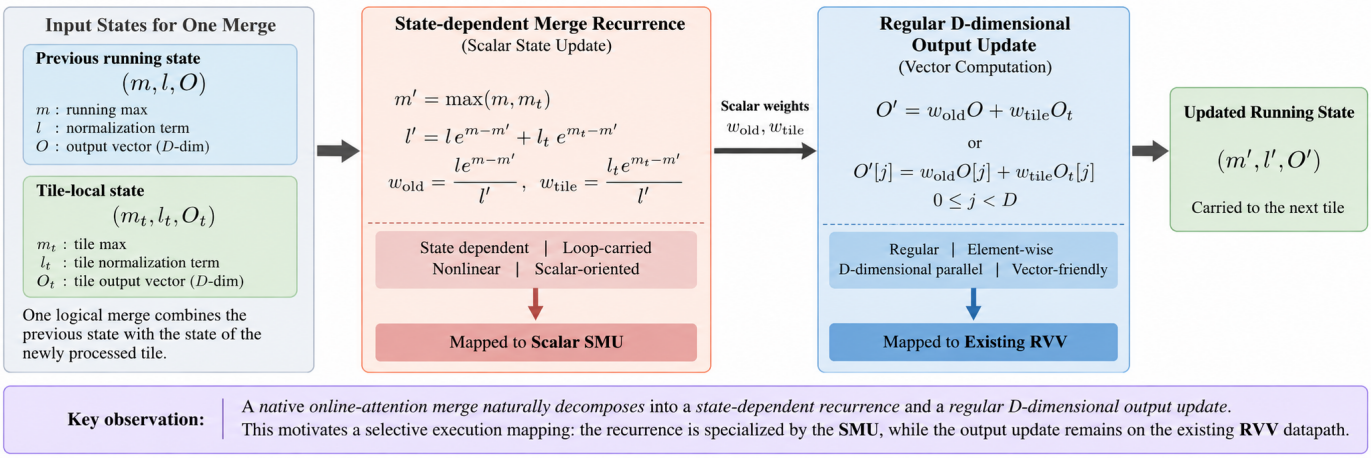


Fig. 1. Execution decomposition of a native online-attention merge. The state-dependent recurrence updates m and l and generates two scalar rescaling weights. The regular D-dimensional output update stays mapped to the existing Spatz datapath.

share the tightly coupled data memory (TCDM). The cluster-local SMU handles the Section II recurrence; Spatz/RVV keeps the regular D-dimensional update.

The online-merge (`OMERGE`) command follows Snitch decode, the per-core accelerator demultiplexer, cluster issue arbiter, and `OMERGE` adapter/SMU. The arbiter records the issuing core, and a response demultiplexer returns completion to that owner. Spatz/RVV instructions take the direct vector path. The SMU is an independent TCDM requester that reads operands and writes scalar state and weights through the shared memory. The default configuration provides 128 KiB of TCDM in 16 banks.

The integration leaves the Spatz/RVV lanes, floating-point units, vector register file, and supported instruction semantics untouched, and introduces no custom RVV instruction. The SMU exchanges scalar state and weights through TCDM, so the RVV routine applies Eqs. (3) and (4) as they are, with no D-wide SMU datapath.

Query, key, and value (Q/K/V) data and working buffers are prepared before the timed Native Core region. The region excludes allocation, input generation, diagnostics, Q/K/V loading, and linear projection. It includes score tiles, local states, first-tile initialization, later merges, RVV updates, and the final output copy. The first tile copies $(m_t, l_t, \mathbf{O}_t)$ into the running state. Each later `OMERGE` processes all N configured query rows for one score tile. For each row it reads the active m, l and tile-local m_t, l_t, writes m', l' to the inactive A/B buffers, and stores both weights in TCDM. After all rows complete, the adapter returns status to the issuing core and commits the A/B selector only when a successful response is accepted; errors leave it unchanged. Spatz/RVV then reads the weights and updates each output row, after which software advances the output pointers. Detailed configuration, invocation, and selector semantics appear in Sections III-C–III-E.

The SMU handles only the scalar recurrence. Score generation, local-state construction, output addressing and update, first-tile initialization, and the final copy stay in software or on Spatz/RVV.

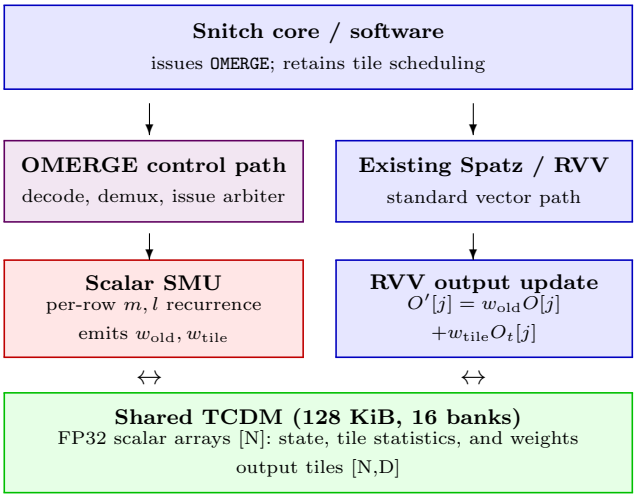


Fig. 2. System integration and HW/SW partition. The Snitch core issues `OMERGE` through the accelerator decode, demultiplexer, and issue arbiter to the Scalar SMU; the adapter returns completion status to the issuing core. The existing Spatz/RVV path performs the regular output update. Both paths access the shared TCDM for row-wise scalar state, weights, and output tiles.

B. Stage-Aware Mixed-Precision SMU

The SMU assigns precision by stage to accommodate the different numerical ranges along the recurrence. The assignment is static and shown in Fig. 3; input values never select precision at runtime. What the SMU computes is a stage-quantized approximation of the exact recurrence in Eq. (2), not exact FP32 arithmetic. We write $\mathrm{RNE}_{16}(\cdot)$, $\mathrm{FP16}(\cdot)$, and $\mathrm{Q16.32}(\cdot)$ for conversion to FP16 round-to-nearest-even, FP16, and the 48-bit unsigned Q16.32 carrier; a hat marks the converted value. The TCDM interface carries m, m_t, l, and l_t in FP32. The maxima and their difference use FP16

round-to-nearest-even:

$$\begin{aligned} \widehat{m} &= \mathrm{RNE}_{16}(m), \qquad \widehat{m}_t = \mathrm{RNE}_{16}(m_t), \\ \widehat{m'} &= \max(\widehat{m}, \widehat{m}_t), \quad \widehat{\Delta}_{\mathrm{old}} = \widehat{m} - \widehat{m'}, \\ \widehat{\Delta}_{\mathrm{tile}} &= \widehat{m}_t - \widehat{m'}. \end{aligned} \tag{5}$$

The exponential (EXP) stage uses one 256-entry signed Q1.14 lookup table (LUT) addressed by

$$i(\Delta) = \lfloor (\Delta + 8) \cdot 32 \rfloor, \qquad \Delta \in [-8, 0), \tag{6}$$

with bin-center codes and no interpolation. The winning contribution has $\Delta \geq 0$ and bypasses the LUT with exact unity, so equal converted maxima yield unity on both paths. Only the losing difference accesses the LUT; the EXP path saturates it only for $|\Delta| > 8$, while Eq. (6) makes $|\Delta| = 8$ access LUT[0]. Denote the factors by $\widehat{s}_{\mathrm{old}}$ and $\widehat{s}_{\mathrm{tile}}$.

The SMU converts FP32 l and l_t to unsigned Q16.32 carriers, denoted $\widehat{l}$ and $\widehat{l}_t$, and computes

$$\widehat{l'} = \widehat{l}\,\widehat{s}_{\mathrm{old}} + \widehat{l}_t\,\widehat{s}_{\mathrm{tile}} \tag{7}$$

The sum in Eq. (7) remains in the unsigned Q16.32 carrier. The denominator crosses an FP16 RNE boundary before reciprocal evaluation. For a normalized FP16-rounded denominator $\widehat{x} = 2^{e-127}(1+\mu)$, where μ is the normalized fractional mantissa, the reciprocal is approximated by

$$\begin{aligned} \frac{1}{\widehat{x}} &\approx 2^{\,127-e}\Big[L_j + (L_{j+1} - L_j) f_j\Big], \\ j &= \lfloor 256\mu \rfloor, \qquad f_j = 256\mu - j. \end{aligned} \tag{8}$$

where L_j is the unsigned Q1.15 code for $1/(1 + j/256)$ and f_j is carried in the low 15 mantissa bits. The reciprocal value from Eq. (8) scales each numerator. Each scaled numerator also crosses the FP16 RNE boundary and returns to Q16.32 before weight computation. One shared 48×16 multiplier produces the old and tile weights sequentially. The reciprocal-LUT path is selected by the `ONLINE_MERGE_MIXED_RECIPROCAL` build definition; its provenance is reported with the evaluation results.

The supported numerical contract requires a positive finite denominator after FP16 conversion and no overflow in fixed-point intermediates. Q16.32 storage does not remove the FP16 normalization boundary. Directed model vectors cover zero numerators, subnormals, and the largest finite FP16 denominator; zero over zero is unsupported. Arbitrary nonfinite inputs and overflow recovery are outside the validated workload contract. EXP truncation and FP16 rounding remain sources of total recurrence error.

The engine encodes $\widehat{m'}$ and $\widehat{l'}$ as FP32 for TCDM writeback. The encoding restores the memory-side carrier, not precision already discarded: the maximum comes from FP16 rounding and the normalization term from Q16.32. Weights become $w = \mathrm{FP32}(\mathrm{FP16}(\widehat{w}))$ through the conversions of Eq. (5) and are consumed by the unchanged FP32 Spatz/RVV update. FP32 therefore carries memory-visible state, Q16.32 carries normalization range, and FP16 plus fixed-point lookup hardware handles the bounded nonlinear stages. All assignments are fixed at design time.

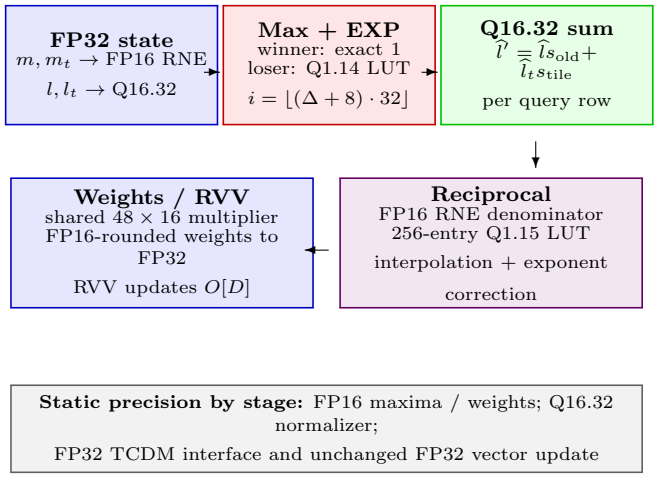


Fig. 3. Stage-aware mixed-precision Scalar SMU datapath. The diagram shows per-row FP32 state, FP16-RNE maxima and reciprocal boundary, Q16.32 normalization, direct-bin EXP lookup, interpolated reciprocal correction, and FP16-to-FP32 merge weights for the unchanged RVV output update.

C. RISC-V Instruction Interface

The programming model separates persistent context setup from recurring execution. Both operations use the RISC-V custom-2 opcode space reserved for implementation-specific extensions [14] and enter the existing Snitch accelerator request path in the Spatz cluster [12]. Their encodings and architectural effects are listed in Table I, with exact decoder patterns in Table II. Neither instruction changes the RVV encoding or the semantics of supported vector instructions.

Table II gives the exact decoder masks, match values, and machine-word expressions. The fixed function bits separate the recurring command from the configuration namespace while keeping both operations in custom-2.

`OMERGE` is a scalar recurrence command, not a vector operation. Its `rs1` and `rs2` fields both name `x0`. Each command applies the scalar recurrence to all N configured rows of one score tile, reading m, l, m_t, and l_t and writing m', l', and the two rescaling weights through TCDM. No D-dimensional output data crosses the instruction interface. The adapter fixes the engine to Mixed Scalar mode, matching the arithmetic used by the MMIO-controlled SMU path; the existing RVV routine performs the output update.

The `rd` field names the status destination. Software must use a nonzero destination to inspect the returned status; `rd=x0` discards the register write. On issue, Snitch reserves that destination in its accelerator scoreboard and forwards the instruction and request identifier through the per-core accelerator demultiplexer and cluster issue arbiter. The adapter permits one outstanding command and follows `IDLE`, `START`, `WAIT`, and `RESP`. In `WAIT`, it waits for the SMU to report DONE or ERROR. In `RESP`, the response path returns the request identifier and status to the issuing core; accepting that response writes `rd` and clears its pending scoreboard entry. A zero status denotes success and a nonzero status denotes an

TABLE I
CUSTOM-2 INSTRUCTION FORMATS AND ARCHITECTURAL EFFECTS.

Instruction	Encoding fields	Register and immediate fields	Architectural effect
OMERGE	R-type; opcode 0x5B; funct7 0000011; funct3 000	rs1=rs2=x0; rd is the status destination	Processes all configured N rows for one score tile, reads and writes each row's scalar recurrence state through TCDM, stores the two weights, and returns completion status.
OMCFG	I-type-like; opcode 0x5B; funct3 001; rs1 supplies the value; rd=x0 imm12=cfg_id		Writes one persistent context field or issues the separate INIT commit operation. It starts no recurrence and returns no register result.

TABLE II
EXACT DECODER PATTERNS FOR THE TWO CUSTOM-2 COMMANDS. THE ADAPTER RECOGNIZES A WORD WHEN ITS BITWISE AND WITH THE MASK EQUALS THE MATCH VALUE.

Command	Mask, match, and machine word
OMERGE	mask 0xFFFFF07F, match 0x0600005B, word 0x0600005B | (rd « 7); instruction[11:7] encodes rd, the completion-status destination, while rs1=rs2=x0.
OMCFG	mask 0x00007FFF, match 0x0000105B, word (cfg_id « 20) | (rs1 « 15) | 0x0000105B; the 12-bit cfg_id occupies bits [31:20] and rs1 supplies the field value in bits [19:15], with rd=x0.

error. The instruction is therefore complete to software only after the core accepts the accelerator response.

The recurring timing comparison uses this complete path: issue, SMU execution and busy time, response commit, and status consumption. It is an inclusive recurrence/control window, not a dispatch-only latency. The separate OMERGE interface comparison holds the SMU arithmetic and RVV update constant while changing how each non-first-tile recurrence is invoked.

D. Runtime-Configurable Context with OMCFG

Addresses and workload parameters need to persist across a sequence of tile merges. Rewriting them around every recurrence would put setup traffic back in the steady-state path. OMCFG writes workload-persistent context once; each later score tile uses one OMERGE invocation to process all configured query rows.

Figure 4 places the four ownership classes on the execution timeline: OMCFG establishes persistent context, OMERGE advances the scalar state, hardware toggles the selector after successful completion, and software updates the output-buffer pointers.

OMCFG uses the same custom-2 opcode as OMERGE and selects funct3 001. The I-type-like encoding places a 12-bit configuration identifier in imm12 and the value in rs1; rd is fixed to x0. This creates no destination scoreboard entry and returns no writeback payload. The adapter accepts a configuration command only while the adapter and SMU engine are idle. The immediate selects a field; it does not carry the field value.

IDs 0x000–0x008 write nine required fields. They specify the physical TCDM base addresses for the two running-state pairs, tile-local m_t/l_t, the two weight arrays, and query-row count N. ID 0x009 selects the separate CTRL/INIT operation; its value bit 0 commits a complete context and resets the selector. Table III lists the mapping and its boundary. Each configured address is the base of a per-row FP32 array; scalar_addr(base,row)=base+(row«2) places row r at byte offset $4r$.

The nine required fields share the canonical generated registers with the legacy MMIO interface. An OMCFG write updates the same register as MMIO, so an MMIO read observes the written value. The adapter tracks writes with a 9'h1FF mask and marks the context dirty. INIT with bit 0 set commits only when all nine required bits are present; success sets cfg_valid and resets the selector to zero. INIT with bit 0 clear does not commit. Rewriting a required field makes another complete INIT necessary before the next merge.

This is runtime configurability over a deliberately narrow context, not a general SMU control ISA. Version 1 exposes no IDs for D, precision or mode, tile size, output-vector addresses, stride, or selector. The adapter fixes Mixed Scalar mode and supplies validation constants for unused scalar-mode fields, including $D = 1$ and stride zero. Workload dimension and output addressing remain software/RVV parameters. A reserved ID performs no register write and invalidates readiness; software must issue a complete INIT before another OMERGE. The interface evaluation therefore separates one-time setup from recurring invocation and the broader Native Core window.

E. Invocation Sequence and State Ownership

The programmed addresses name two physical running-state buffers. A one-bit hardware selector identifies which pair supplies the current scalar state and which pair receives the next state, avoiding per-merge address updates in software. Figure 4 shows the lifetime: software initializes the first tile and output state, writes the nine required fields, commits INIT, then issues one OMERGE for each later tile. After a successful response, the existing RVV routine updates the output and software swaps its output-buffer pointers.

The selector operates on two TCDM-resident buffer pairs $A = (A_m, A_l)$ and $B = (B_m, B_l)$, each holding one FP32 maximum and one FP32 normalizer per query row $0 \le r < N$.

Selector zero selects A as the source and B as the destination. Selector one selects B as the source and A as the destination. The selector changes the physical m/l addresses supplied to the SMU. The buffers stay in TCDM and retain their contents.

Software handles the first tile as initialization. It writes the initial running scalar state into A and initializes the output buffer in software, so no OMERGE runs for that tile. A complete

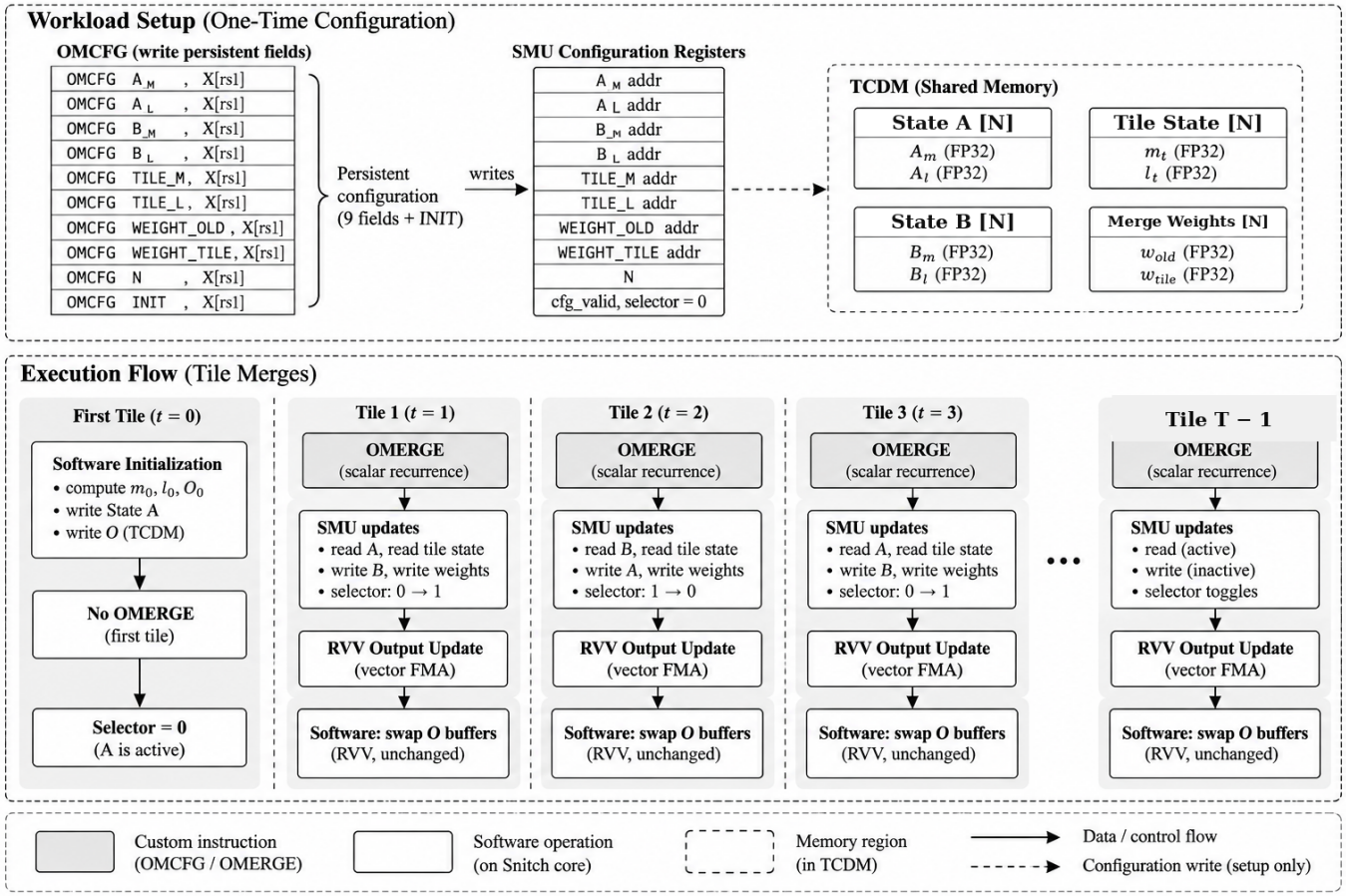


Fig. 4. `OMCFG`/`OMERGE` programming model. One-time `OMCFG` writes establish persistent scalar context and reset the selector. Each later score tile uses one `OMERGE` command to process the configured N query rows before the unchanged RVV output update. With tile size $B = 4$, a workload of N rows has $T = N/B$ tiles, and the final tile is $T - 1$. The dashed setup path writes context addresses, not TCDM data. Hardware toggles only the A/B scalar-state selector; software swaps output buffers. The selector changes after the issuing core accepts a successful response.

TABLE III
OMCFG CONFIGURATION-ID MAP. IDS 0–8 ARE THE REQUIRED CONTEXT FIELDS; ID 9 IS THE SEPARATE INIT CONTROL OPERATION.

cfg_id	Field	Value or address	Purpose and write semantics
0x000	A_M	TCDM address	Per-row running maximum array in state buffer A
0x001	A_L	TCDM address	Per-row running normalizer array in state buffer A
0x002	B_M	TCDM address	Per-row running maximum array in state buffer B
0x003	B_L	TCDM address	Per-row running normalizer array in state buffer B
0x004	TILE_M	TCDM address	Per-row tile-local maximum array m_t
0x005	TILE_L	TCDM address	Per-row tile-local normalizer array l_t
0x006	WEIGHT_OLD	TCDM address	Base of per-row w_{old} array
0x007	WEIGHT_TILE	TCDM address	Base of per-row w_{tile} array
0x008	N	Query-row count	Rows processed by each `OMERGE` invocation
0x009	CTRL/INIT	Bit 0 = 1	Commit all required fields and reset the A/B selector
0x00A–0xFFF	Reserved	No register write	Unsupported IDs invalidate readiness until a complete INIT

INIT leaves the selector at zero. The first later merge reads A and writes B; subsequent merges alternate the direction.

The adapter commits a selector transition only when the issuing core accepts a zero-error accelerator response. Issue and SMU start leave the selector unchanged. An incomplete or dirty context returns a nonzero status without asserting engine start. An SMU error also returns a nonzero status. Both cases leave the selector at its previous value, so a failed command cannot redirect the next successful merge.

The selector owns only the per-row scalar m/l state. Software continues to maintain the two D-dimensional output-buffer pointers, and the unchanged RVV routine performs the output update after each successful recurrence. Software also owns tile addresses and output pointers. The evaluated contract uses one controlling core and one active context. Request-owner routing does not provide per-core context isolation. Concurrent configuration writers, interrupts, cancellation, and fault recovery are outside the validated scope. Software consumes successful completion before the RVV update; the two anchor tests establish output equivalence, not a general proof of memory ordering under concurrent execution.

IV. Evaluation

A. Experimental Setup and Measurement Scope

Performance results come from cycle-level simulation of the Spatz RISC-V vector cluster. The Verilated model uses Verilator 5.034 and the default cluster configuration with 128 KiB of 16-bank TCDM. Cycle counters bracket each measured stage explicitly, and every matched comparison runs on the same frozen cluster configuration and software toolchain.

N8/D32 and N16/D64 are the two anchor workloads. N24/D64 and N16/D128 test whether the observed behavior survives a change in sequence length or output dimension. All native-attention cases use tile size $B = 4$ and seed one. The first tile initializes the running state directly and needs no merge. Each query row therefore has $T - 1$ later-tile merges: one at N8/D32, three at N16/D64, five at N24/D64, and three at N16/D128. Each later score tile issues one `OMERGE`, which processes all configured N rows.

The design accelerates only part of native online attention, so we report three timing scopes. The B2R software *recurrence* window covers the scalar state-dependent update that produces m', l', w_{old}, and w_{tile}. For the matched MMIO-controlled SMU path, the reported recurrence window also includes per-merge register programming, clear/start, BUSY/DONE polling, and SMU execution. It therefore measures the complete recurrence invocation, including control and engine-busy time, rather than arithmetic-only latency. The *merge* window adds the corresponding RVV output update and the merge-side orchestration. The *Native Core* window adds score-tile computation, tile-local state construction, first-tile handling, all later merges, and the final output handling. Allocation, input generation, diagnostics, Q/K/V loading, and linear projection stay outside all three.

The comparisons form independent matched families. The arithmetic-offload family compares a scalar-software recurrence plus RVV (B2R) with the MMIO-controlled Mixed Scalar SMU plus the same RVV update. The recurring-interface family holds SMU arithmetic and RVV common and compares per-merge MMIO control with `OMERGE`. The configuration family holds `OMERGE` common and compares one-time MMIO configuration with `OMCFG`. Full SMU appears only in a separate historical ablation.

Each family uses its own matched baseline and frozen implementation. Absolute cycles across families do not form an optimization trajectory, and no matched comparison between B2R and the final `OMCFG`/`OMERGE` interface is available. B2R implements the same mathematical recurrence and shares the RVV routine, but its internal numerical formats differ from the SMU's. The recurrence ratio therefore includes both hardware specialization and numerical-implementation differences, which are not separated here. B2R is not an optimized software-library baseline. All native cases use one seed, and the interface families contain one formal run per path and shape. These results establish deterministic prototype behavior, not model accuracy, multi-head scaling, or statistical generality. Speedups are cycle-count ratios without an achieved-frequency assumption.

B. Native Online Attention Performance

The native comparison holds the output update constant to measure recurrence offload. B2R is the matched software baseline: it executes the recurrence in scalar software and shares the SMU path's RVV output routine. Figure 5 summarizes the three timing scopes.

At N8/D32 the recurrence falls from 12,368 to 1,451 cycles, an 8.524× speedup. The merge window falls from 13,204 to 2,318 cycles, or 5.696×. Native Core cycles drop from 85,262 to 73,706, a 1.157× speedup and a 13.554% cycle reduction.

At N16/D64 the recurrence falls from 68,994 to 4,926 cycles, or 14.006×. The merge window falls from 77,037 to 13,060 cycles, 5.899×. Native Core drops from 571,374 to 507,918 cycles, a 1.125× speedup and an 11.106% reduction.

N24/D64 reaches 17.784× at the recurrence scope, 6.275× at the merge scope, and 1.140× at the Native Core scope. N16/D128 reaches 13.737×, 4.144×, and 1.066× at those same three scopes. Across all four workloads the measured ranges are 8.524–17.784× for recurrence, 4.144–6.275× for merge, and 1.066–1.157× for Native Core.

Each wider scope contains a larger share of work this design leaves untouched: score construction, tile-local state generation, and the regular vector output update all stay on the programmable path. Native Core therefore improves less than the recurrence, and the recurrence result is not an attention- or model-level speedup.

C. Numerical Fidelity and Standalone SMU Cost

The stage-aware mixed-precision SMU agrees closely with the matched B2R software recurrence and reduces standalone mapped area. Across the four native-attention workloads, the final Mixed Scalar SMU reaches cosine similarities of 0.999970724 to 0.999984751. No output is nonfinite in these cases.

Table IV(a) compares the native cases with B2R software outputs; metrics are computed in float64 from the stored FP32 results. FP32 is the storage type of both paths, not an independent oracle. A separate numerical stress sweep drops to a minimum cosine similarity of 0.999873853, which rules out any global claim above 0.9999. Sweeping all 31,743 positive finite FP16 denominators exhaustively gives a maximum reciprocal relative error of 6.6966×10^{-5}. The separate host-model scan covers random-score and single-head Q/K/V cases, lengths 128, 256, 512, and 1024, and dimensions 64 and 128 (16 combinations), with tile size 32 and base seed 20260823. It is distinct from the tile-four RTL native-performance experiments. The selected reciprocal scan has minimum replacement-versus-division cosine 0.999999885, while the optimized-versus-FP32 minimum is 0.999873853. These model results do not establish RTL performance at those lengths.

Stage-aware precision is measured with matched standalone Yosys/ABC synthesis against the Nangate45 typical library. Replacing the Legacy arithmetic with mixed precision, and keeping division, cuts mapped area from 72,399.082 to 52,000.074 units, a 28.176% reduction. The final reciprocal

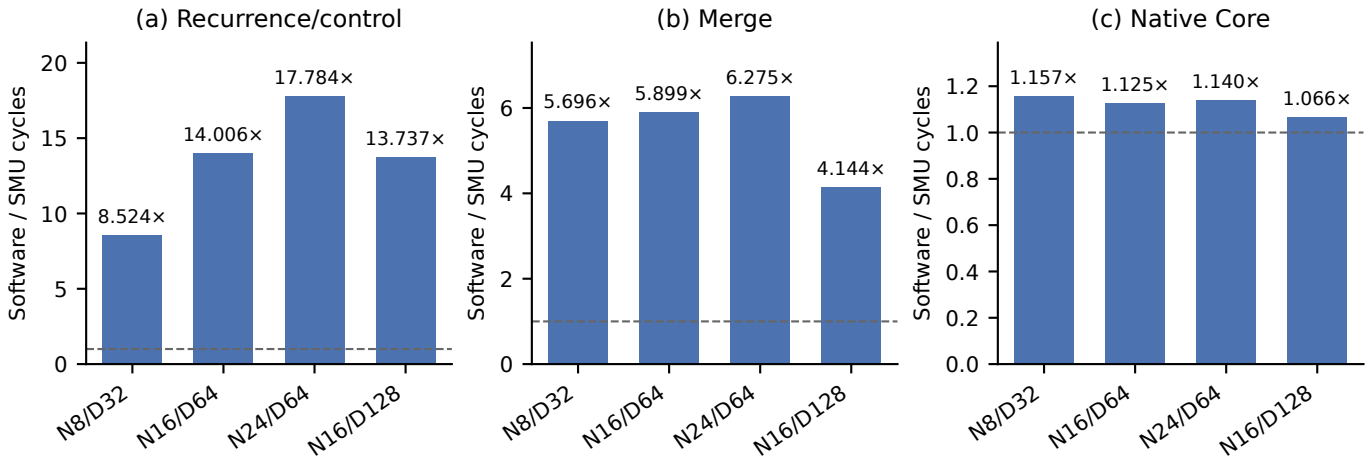


Fig. 5. Native online-attention speedup across recurrence, merge, and Native Core measurement scopes. The MMIO-controlled SMU recurrence window includes per-merge configuration, clear/start, polling, and SMU execution. Ratios use matched software and SMU cycle counts, tile size four, and seed one. Each bar represents one deterministic case; no uncertainty interval is estimated. Panel scales differ.

TABLE IV
NUMERICAL AGREEMENT WITH THE B2R SOFTWARE RECURRENCE AND STANDALONE SMU COST. AREA IS FROM THE AREA-MAPPING FLOW AND THE DELAY PROXY FROM A SEPARATELY TIMING-CONSTRAINED MAPPING OF THE SAME DESIGNS; THE TWO COLUMNS DO NOT DESCRIBE ONE NETLIST.

(a) Agreement with B2R software outputs

Shape	MAE	Cosine similarity
N8/D32	2.793084×10^{-4}	0.999984541
N16/D64	4.665494×10^{-4}	0.999970724
N24/D64	3.736880×10^{-4}	0.999980879
N16/D128	5.631592×10^{-4}	0.999984751

(b) Standalone synthesis

Design	Mapped area	Area reduction vs. Legacy	Delay proxy
Legacy Scalar	72,399.082	n/a	12.518 ns
Mixed+Division	52,000.074	28.176%	14.620 ns
Mixed+Reciprocal	52,407.586	27.613%	15.540 ns

implementation occupies 52,407.586 units. That is 27.613% below Legacy and 0.784% above Mixed+Division.

Lower area does not mean better timing. The longest mapped register-to-register delay proxy rises from 12.518 ns for Legacy to 14.620 ns for Mixed+Division and 15.540 ns for Mixed+Reciprocal. These values are pre-layout timing proxies only, and no design meets the 1.5 ns target. Cluster synthesis exhausted resources during techmap, so integrated area, frequency, and power remain unmeasured; the area comparison is standalone.

D. HW/SW Partition Ablation

The final architecture keeps the D-dimensional output update on RVV and omits a dedicated vector datapath in the SMU. We examine this choice with a separately frozen Full SMU/A2 experiment. This ablation predates the final `OMERGE`/`OMCFG` implementation, so its absolute area and cycle values remain within their own matched historical epoch.

There, the selective design combines scalar recurrence offload with the RVV output update, and Full SMU additionally moves the dimension-dependent output update into the accelerator. Standalone area rises from 77,103.292 to 114,713.298 units, a 48.779% increase. The historical mapped delay proxy also rises, from 12.34285 to 13.20260 ns.

TABLE V
HISTORICAL SELECTIVE-VERSUS-FULL-OFFLOAD ABLATION. THE CYCLE WINDOW AND IMPLEMENTATION EPOCH ARE COMMON WITHIN EACH ROW AND SEPARATE FROM THE PRIMARY NATIVE-ATTENTION EXPERIMENT.

Shape	Selective	Full SMU	Full/Selective
N12/D64	4,128	7,704	1.866×
N32/D128	12,866	34,728	2.699×
N40/D128	15,733	43,206	2.746×

These are historical workload measurements, not full BERT, Mistral, or Qwen model runtimes. Full SMU is slower in all three matched cases. These values stay out of the final native-attention results. Together with the area comparison they support the selective partition: regular D-dimensional work stays on the RVV datapath instead of being duplicated inside the SMU.

E. RISC-V Instruction-Interface Evaluation

The ISA interface is evaluated in two stages: recurring per-merge invocation first, then one-time workload configuration with `OMERGE` held common.

A1-MMIO and A1-ISA/`OMERGE` use the same Mixed Scalar SMU arithmetic, RVV output routine, workload, tile size, and

TABLE VI
ISA INTERFACE EVALUATION. IN (A), BOTH PATHS USE THE SAME MIXED SCALAR SMU ARITHMETIC AND RVV ROUTINE, SO SMU BUSY CYCLES INSIDE THE RECURRENCE AND MERGE WINDOWS CANCEL OR NEARLY CANCEL (N16/D64 DIFFERS BY 4 CYCLES). THE CONTROL ROW SUBTRACTS THE COMMON BUSY CYCLES AND LEAVES LAUNCH, POLL, AND COMMIT ACTIVITY.

(a) `OMERGE` per-merge effect (cycles)

Shape	Window	MMIO	`OMERGE`	Reduction
N8/D32	Ctrl. excl. busy	1,261	40	96.828%
	Rec./Ctrl.	2,069	848	59.014%
	Merge	2,926	1,706	41.695%
	Setup-incl. Core	74,392	73,766	0.841%
N16/D64	Ctrl. excl. busy	3,577	81	97.736%
	Rec./Ctrl.	8,429	4,929	41.523%
	Merge	16,548	12,915	21.954%
	Setup-incl. Core	510,991	508,702	0.448%

(b) `OMCFG` setup effect

Shape	MMIO setup	`OMCFG` setup	Setup reduction	Native Core Δ
N8/D32	991	659	33.50%	0
N16/D64	986	632	35.90%	+12

numerical semantics. They differ only in how each non-first-tile recurrence is triggered. Table VI(a) reports the matched windows in cycles for both anchors.

The inclusive recurrence/control window contains issue and control activity, SMU execution and busy time, response commit, and status consumption, so its reduction measures more than dispatch latency. Most of that window is SMU busy time, which both paths pay at equal or nearly equal cost. Subtracting it isolates the control activity around the recurrence: 1,261 cycles fall to 40 at N8/D32 and 3,577 fall to 81 at N16/D64. Widening the scope adds work the interface does not touch, so the reported reduction falls to 41.695% and 21.954% at the merge scope, and to 0.841% and 0.448% when the one-time ISA setup enters the Native Core window.

With `OMERGE` held common, we compare MMIO-config+`OMERGE` against `OMCFG`-config+`OMERGE`. Both variants execute the same `OMERGE`-based Native Core. They differ only in the transport of nine configuration fields and one INIT operation before core execution. For N8/D32, setup decreases from 991 to 659 cycles, a 33.50% reduction. For N16/D64, it decreases from 986 to 632 cycles, a 35.90% reduction. Native Core execution is identical for N8/D32 and differs by only 12 cycles for N16/D64; the corresponding setup-inclusive totals fall by 0.451% and 0.067%, respectively. Table VI summarizes both comparisons.

Both interface comparisons preserve functional behavior. The `OMERGE` and MMIO-controlled paths produce bit-identical final outputs for both anchor workloads. N8/D32 issues one `OMERGE` and N16/D64 issues three, matching the expected number of non-first-tile merges and selector transitions; each command processes all configured query rows. The final output hashes are `0x9c7cdedc` for N8/D32 and `0xfd0d2cff` for N16/D64. The matched outputs have zero pairwise MAE and stable relative error, a cosine similarity of one, and no nonfinite values. The recurring-interface experiment did not archive intermediate m/l and weight arrays, so we make no intermediate bit-identity claim.

Both configuration routines are present in each binary. A data-resident `volatile` path selector has a different initial value in the two builds and selects MMIO or OMCFG at runtime. Consequently, the MMIO-config and `OMCFG`-config binaries have byte-identical `.text` within each workload and produce word-for-word identical final outputs. They also establish identical persistent configuration state after INIT, including all nine fields and selector zero. `OMCFG` therefore reduces measured one-time setup cost with no measured systematic steady-state Native Core benefit.

F. Explicit-P Negative Control

The final experiment tests the SMU as a general replacement for row-wise Softmax in an explicit probability-matrix dataflow. The surrounding quantized-attention path is held fixed; only the explicit-P software Softmax backend is swapped for the Mixed SMU backend.

For N8/D32, Mixed SMU increases the explicit-P Softmax window from 12,502 to 14,812 cycles, making it 18.477% slower than software. For N16/D64, the window changes from 48,595 to 48,534 cycles, a 0.126% improvement. The wider attention windows remain slightly slower with the SMU in both cases.

The five-stage Attention Total changes from 839,111 to 841,597 cycles for N8/D32, a 0.296% increase, and from 6,767,387 to 6,768,406 cycles for N16/D64, a 0.015% increase. The complete Mixed Softmax window includes P initialization, SMU launch and polling, the RVV probability update, and swap/clear orchestration. The five-stage Attention Total sums arithmetic terms; it does not come from an independent timing counter.

These results contrast with the native online-attention results in Section IV-B. As a generic row-wise Softmax replacement the SMU offers no consistent benefit in these two cases. The positive native results and this negative control delimit the measured applicability of the implementation; they do not prove that other Softmax implementations cannot benefit.

V. DISCUSSION

A. The gain follows the chosen execution boundary

The measured benefit is largest at the recurrence boundary and narrows as the window includes work left on the programmable path. Score-tile computation and the regular D-element output update remain outside the scalar accelerator; the Native Core result therefore reflects this partition's effect on the measured kernel, with Q/K/V loading and linear projections excluded.

Together, the native and explicit-P measurements support a boundary-specific result: the SMU targets the persistent scalar recurrence of native online attention, not a generally faster Softmax backend.

B. The instruction interface exposes the partition to software

Software writes the nine supported context fields with `OMCFG` and commits readiness, then invokes `OMERGE` for each later score tile. Hardware processes all configured query rows

and tracks the alternating scalar-state selector; software retains tile scheduling, output-buffer pointers, and RVV execution. The core's accelerator path exposes this stateful operation without a custom vector instruction or a duplicated output datapath.

The measurements do not support treating the instruction interface as the source of the main speedup. `OMERGE` reduces the inclusive recurrence/control window by 41.5–59.0% on two anchors, but that window includes SMU busy time and response handling; it is not dispatch-only latency. `OMCFG` reduces one-time setup by 33.5–35.9%; setup-inclusive totals change by 0.451% and 0.067%. These are control-plane results that complement the selective dataflow result. They do not establish a material steady-state Native Core gain from either instruction in isolation.

This interface is distinct in granularity from prior RISC-V nonlinear extensions such as VEXP's scalar and packed-SIMD exponential instructions. The present commands expose the online-merge recurrence and its persistent state, not a general exponential primitive. That difference narrows the positioning; it does not imply that online normalization, Softmax hardware, or RISC-V nonlinear acceleration is new.

C. Numerical and evaluation limits

Matched cluster synthesis exhausted resources during techmap, so integrated area, frequency, and power were not measured. B2R is matched recurrence software sharing the RVV routine, not an established optimized library baseline.

VI. Comparison with Prior Work

Table VII places this work among representative RISC-V and AI acceleration approaches; the subsections below give the detail behind each row.

A. Softmax and Nonlinear Acceleration

Once matrix multiplication is well optimized, nonlinear operators such as exponentiation, Softmax, and GELU become the bottleneck in Transformer workloads. Prior work attacks this at several granularities, from arithmetic ISA extensions and low-cost reconfigurable nonlinear cores [18], [19] to dedicated nonlinear accelerators.

VEXP puts an approximate BFloat16 exponential unit inside the floating-point unit of Snitch RISC-V cores and exposes it through scalar `FEXP` and packed-SIMD `VFEXP` instructions [8]. Its Softmax kernel pairs that exponential with the existing Snitch execution mechanisms, and reports up to 162.7× lower Softmax latency and an 8.2× gain for FlashAttention-2 in the GPT-2 configuration. VEXP is a fine-grained approach: it adds the costly nonlinear primitive to the programmable datapath.

SoftEx takes the other route and accelerates the whole nonlinear operator in dedicated BFloat16 hardware [7]. Its datapath covers approximate exponentiation, accumulation, inversion, and normalization, and sits in a RISC-V cluster next to a matrix engine. The extended design also supports GELU and reports up to 10.8× Softmax acceleration, which shows what nonlinear hardware beside an already accelerated matrix path can buy.

Our comparison focuses on a different exposed interface: persistent normalized state and per-merge control. In the explicit-P negative control in Section IV-F, the SMU does not consistently improve row-wise Softmax. The measured benefit appears in the native path, where scalar state persists across online merges and the D-dimensional output update remains on RVV.

B. RISC-V Domain-Specific Acceleration

RISC-V's extensibility lets domain-specific hardware stay under software control without a separate programming model. Systems so far span tightly integrated functional-unit extensions [10], [8], cluster-local coprocessors [15], [7], and vector-processor extensions [11], [13], [12].

Spatz provides the programmable vector substrate used here: a compact RISC-V vector processor for tightly coupled shared-memory clusters, with regular vector execution and a comparatively small vector register file [12]. Our design keeps that vector datapath for the regular output update and leaves RVV execution semantics untouched.

TOP-V is another example of selective specialization [13]. Mixture-of-Experts routing needs a branch- and dependency-heavy Top-K operation, so TOP-V adds a configurable sorting unit to the Ara RISC-V vector processor and exposes it as a custom `VTOPK` instruction. The reported implementation reaches up to 5.24× over an RVV-based sort. Irregular, dependency-heavy work can justify dedicated hardware even when the surrounding computation already maps well to a vector processor.

Cluster-local accelerators take a complementary route. RedMulE is a tightly coupled matrix engine for floating-point GEMM and GEMM-like operations in PULP RISC-V clusters [15]. It attaches a dedicated engine to the shared cluster memory and lets the programmable cores coordinate it.

Across these systems, specialization sits in a functional unit, a cluster coprocessor, or the vector processor; the SMU interface exposes online-merge state and alternating scalar-state ownership.

The SMU issues its own TCDM requests, so standard Spatz/RVV execution is unaffected. The custom interface separates workload-persistent configuration (`OMCFG`) from recurring execution (`OMERGE`), and only the alternating scalar-state selector lives in hardware, so specialization stays outside the regular vector datapath without repeated software control on every merge.

C. HW/SW Co-Design for AI Acceleration

Gemmini integrates a configurable systolic array with a RISC-V host over RoCC and evaluates it through a full software stack [16]. Deeploy maps networks across heterogeneous RISC-V microcontrollers, generating optimized C for RV32 cores, machine-learning extensions, and an NPU under tight memory constraints [17]. Both target complete accelerators or compiler mappings; our interface specializes a recurrence over persistent scalar state within a vector cluster. For online attention, software retains score construction, tile management, first-tile initialization, output-buffer ownership,

TABLE VII
POSITIONING OF REPRESENTATIVE RISC-V AND AI ACCELERATION APPROACHES. REPORTED RESULTS RETAIN EACH ORIGINAL WORK'S SCOPE AND ARE NOT DIRECTLY COMPARABLE ACROSS ROWS.

Work	Hardware path and vector relation	Reported result
VEXP [8]	Snitch FPU with custom scalar/packed `FEXP`/`VFEXP` for BF16 exponentiation	162.7× lower Softmax latency; 8.2× FlashAttention-2 speedup
SoftEx [7]	RISC-V cluster HWPE accelerates Softmax/GELU through control and streaming interfaces	10.8× Softmax speedup; 1.58× end-to-end ViT throughput
TOP-V [13]	Ara vector processor with custom `VTOPK` sorting instruction	5.24× over RVV-based sorting
RedMulE [15]	Cluster-local HWPE-style GEMM engine sharing cluster memory	15× GEMM; up to 62× GEMM-Op over software
Gemmini [16]	RoCC-attached systolic array with a full software stack	Full-stack DNN evaluation; orders-of-magnitude over CPU baselines
Deeploy [17]	Compiler maps SLM execution across RV32 cores, ML extensions, and an NPU	340 token/s at 490 μJ/token (TinyStories SLM)
This work	Cluster-local Scalar SMU with `OMCFG`/`OMERGE` specializes the recurrence; Spatz/RVV retains the output update	Up to 17.784× recurrence, 6.275× merge, and 1.157× Native Core (MMIO-controlled SMU) over the B2R software baseline

and RVV, while the SMU handles the scalar recurrence and its alternating state selector.

FPGA attention accelerators span multi-FPGA text generation, dense or compiler-generated datapaths, training accelerators, and sparsity-aware designs [20], [21], [22], [23], [24]. Edge-MoE applies task-level sparsity to a multi-task vision-transformer accelerator, while the TinyML study evaluates a heterogeneous deployment flow [25], [26]. Their evaluations range from accelerator and operator measurements to deployment; our question is which part of online merge a programmable vector cluster should own.

The reported results in Table VII are quoted only to indicate each work's evaluation scope.

VII. CONCLUSION

We present selective HW/SW mapping for native online attention on a RISC-V vector cluster: a cluster-local Scalar Online-Merge Unit (SMU) handles the state-dependent recurrence, while Spatz/RVV retains the regular D-element output update. Stage-aware mixed precision and the `OMCFG`/`OMERGE` interface separate nonlinear arithmetic, persistent configuration, and the alternating scalar-state selector.

Across matched MMIO-controlled native workloads, the design reaches an 8.524–17.784× recurrence speedup; the measured Native Core gain is smaller, and its window excludes Q/K/V loading and linear projections. Standalone mapped area falls 27.613% versus the legacy scalar design, while the pre-layout delay proxy rises. In two explicit-P cases, the SMU gives no consistent benefit as a general row-wise Softmax replacement, supporting a dataflow-specific interpretation of the native online-merge results.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[2] M. Milakov and N. Gimelshein, "Online normalizer calculation for Softmax," arXiv preprint arXiv:1805.02867, 2018, doi: 10.48550/arXiv.1805.02867.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.

[4] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 68 658–68 685, doi: 10.52202/079017-2193. [Online]. Available: https://papers.nips.cc/paper_files/paper/2024/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html

[6] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient Softmax for transformers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 469–474, doi: 10.1109/DAC18074.2021.9586134. [Online]. Available: https://doi.org/10.1109/DAC18074.2021.9586134

[7] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated Softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.

[8] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated Softmax computation in transformers," in *Proc. IEEE Symp. Computer Arithmetic (ARITH)*, 2025, pp. 37–44, doi: 10.1109/ARITH64983.2025.00016.

[9] C. Peltekis, K. Alexandridis, and G. Dimitrakopoulos, "Reusing Softmax hardware unit for GELU computation in transformers," in *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, 2024, pp. 159–163, doi: 10.1109/AICAS59952.2024.10595882. [Online]. Available: https://doi.org/10.1109/AICAS59952.2024.10595882

[10] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads," *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021, doi: 10.1109/TC.2020.3027900. [Online]. Available: https://doi.org/10.1109/TC.2020.3027900

[11] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multiprecision floating-point support in 22-nm FD-SOI," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020, doi: 10.1109/TVLSI.2019.2950087. [Online]. Available: https://doi.org/10.1109/TVLSI.2019.2950087

[12] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025, doi: 10.1109/TCAD.2025.3528349.

[13] Q. Chen, C. Wang, Z. Wang, and J. Lin, "Top-V: A flexible and programmable Top-K acceleration framework based on the RISC-V ISA," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2026.

[14] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, 2019, document Version 20191213.

[15] Y. Tortorella, L. Bertaccini, L. Benini, D. Rossi, and F. Conti, "RedMulE: A mixed-precision matrix–matrix operation engine for flexible and energy-efficient on-chip linear algebra and TinyML training acceleration," *Future Gener. Comput. Syst.*, vol. 149, pp. 122–135, 2023.

[16] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, 2021, pp. 769–774.

[17] M. Scherer *et al.*, "Deeploy: Enabling energy-efficient deployment of small language models on heterogeneous microcontrollers," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 4009–4020, 2024.

[18] Y. Li, W. Cao, X. Zhou, and L. Wang, "A low-cost reconfigurable nonlinear core for embedded DNN applications," in *2020 International Conference on Field-Programmable Technology (ICFPT)*, 2020, pp. 35–38, doi: 10.1109/ICFPT51103.2020.00014. [Online]. Available: https://doi.org/10.1109/ICFPT51103.2020.00014

[19] Q.-X. Wu, S.-S. Teng, M.-D. Shieh, C.-T. Huang, and J.-M. Lu, "A low-complexity and reconfigurable design for nonlinear function approximation in transformers," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 72, no. 12, pp. 1882–1886, 2025, doi: 10.1109/TCSII.2025.3585109. [Online]. Available: https://doi.org/10.1109/TCSII.2025.3585109

[20] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630, doi: 10.1109/MICRO56248.2022.00051. [Online]. Available: https://doi.org/10.1109/MICRO56248.2022.00051

[21] E. Kabir, M. A. Kabir, A. R. J. Downey, J. D. Bakos, D. Andrews, and M. Huang, "FAMOUS: Flexible accelerator for the attention mechanism of transformer on UltraScale+ FPGAs," in *2024 International Conference on Field Programmable Technology (ICFPT)*, 2024, pp. 1–2, doi: 10.1109/ICFPT64416.2024.11113430. [Online]. Available: https://doi.org/10.1109/ICFPT64416.2024.11113430

[22] P. Plagwitz, F. Hannig, and J. Teich, "TRAC: Compilation-based design of transformer accelerators for FPGAs," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022, pp. 17–23, doi: 10.1109/FPL57034.2022.00015. [Online]. Available: https://doi.org/10.1109/FPL57034.2022.00015

[23] H. Shao, J. Lu, M. Wang, and Z. Wang, "An efficient training accelerator for transformers with hardware-algorithm co-optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 11, pp. 1788–1801, 2023, doi: 10.1109/TVLSI.2023.3305569. [Online]. Available: https://doi.org/10.1109/TVLSI.2023.3305569

[24] H. Wang, K. Wang, Y. Zhang, and J. Yu, "FAS-Trans: Fully exploiting FFN and attention sparsity for transformer on FPGA," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9, doi: 10.1145/3676536.3676743. [Online]. Available: https://doi.org/10.1145/3676536.3676743

[25] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, "Edge-MoE: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 01–09, doi: 10.1109/ICCAD57390.2023.10323651. [Online]. Available: https://doi.org/10.1109/ICCAD57390.2023.10323651

[26] P. Wiese, G. İslamoğlu, M. Scherer, L. Macan, V. J.-B. Jung, A. Burrello, F. Conti, and L. Benini, "Toward attention-based TinyML: A heterogeneous accelerated architecture and automated deployment flow," *IEEE Design & Test*, vol. 42, no. 5, pp. 63–72, 2025, doi: 10.1109/MDAT.2025.3527371. [Online]. Available: https://doi.org/10.1109/MDAT.2025.3527371